Rapport de projet

Atelier de Génie Logiciel

SMART DAILY PLANNER

Application de Planification Intelligente pour Étudiants Tunisiens

---

Réalisé par :

Alae Benelmekki A07

Wissem Amiri A07

Ghofrane Fitouri A07

Samar Dhouib A01

Table des matières

1. Fiche Projet 3

1.1Description du projet 3

1.2 Problème résolu 3

1.3 Cible (utilisateurs visés) 3

1.4 Acteurs du système 4

1.5 Fonctionnalités principals 4

2.Product backlog 4

2.1User Stories (10 US) 4

3.Sprint Backlogs 5

3.1 sprint planning 5

3.2 Sprint2 planning 5

4.Modelisation UML 6

4.1 Diagramme de Cas d'Utilisation (DCU) 6

Diagramme de Classes (DC) 7

Diagrammes de Séquence (DS) 7

5. MAQUETTES FIGMA (UI/UX) 9

..... Error! Bookmark not defined.

..... Error! Bookmark not defined.

6. CODE - APPROCHE HYBRIDE AGL + IA 12

6.1 Architecture du Code 12

6.2 Approche Hybride pour Chaque Membre 12

6.3 Processus IA Assistée (Exemple) 13

7. TABLEAU COMPARATIF AGL vs IA 14

8. SPRINT REVIEWS & RÉTROSPECTIVES 15

8.1 Sprint 1 Review 15

8.2 Sprint 1 Rétrospective 15

8.3 Sprint 2 Planning (À faire) 15

9. BILAN INDIVIDUEL 16

Équipe

Membre	Rôle Scrum	Fonctionnalité	Branche Git
Samar Dhouib	Product Owner + Dev	Inscription & Profil Utilisateur Intelligent	feature/auth-profile
Maamoun Khedher	Scrum Master + Dev	Gestion des Tâches (CRUD) et priorités	feature/task-manager
Ghofrane Fitouri	Développeur	Moteur d'Analyse Comportementale & Contraintes	feature/behavior-engine
Wissem Amiri	Développeur	Planificateur Intelligent	feature/smart-scheduler
Alae Benelmekki	Développeur	Tableau de Bord & Statistiques	feature/dashboard-stats

1. Fiche Projet

1.1Description du projet

Smart Daily Planner est une application de planification intelligente qui aide les étudiants tunisiens à organiser leurs tâches quotidiennes en générant automatiquement un emploi du temps optimisé. L'application prend en compte les créneaux de disponibilité, les priorités des tâches, le niveau d'énergie de l'utilisateur et les contraintes de transport. Elle permet un ajustement dynamique du planning en cas d'imprévus (retards, changements d'emploi du temps)

1.2 Problème résolu

Les étudiants tunisiens ont du mal à organiser leur temps entre cours, révisions, activités personnelles et contraintes de transport. La planification manuelle est chronophage et inefficace.

1.3 Cible (utilisateurs visés)

2. Étudiants universitaires tunisiens
3. Élèves du secondaire

1.4 Acteurs du système

Acteur	Description
Utilisateur	Étudiant connecté qui gère ses tâches et son planning
Administrateur	Gère les règles de planification et les paramètres système
Visiteur	

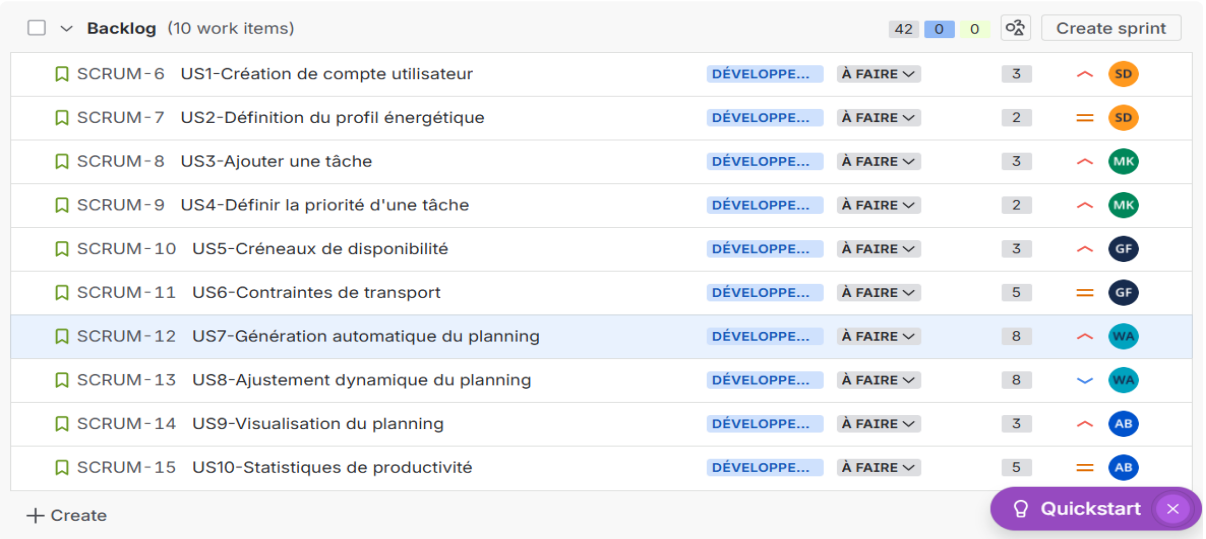
1.5 Fonctionnalités principals

2.Product backlog

Membre	Fonctionnalité
Samar Dhouib	Inscription & Profil Utilisateur Intelligent
Maamoun Khedher	Gestion des Tâches (CRUD) et priorités
Ghofrane Fitouri	Moteur d'Analyse Comportementale & Contraintes
Wissem Amiri	Planificateur Intelligent
Alae Benelmekki	Tableau de Bord & Statistiques

2.1User Stories (10 US)

US	Titre	Priorité	Story Points	Membre
US#1	Création de compte utilisateur	Must	3	Samar
US#2	Profil énergétique	Should	2	Samar
US#3	Ajouter une tâche	Must	3	Maamoun
US#4	Priorité d'une tâche	Must	2	Maamoun
US#5	Créneaux de disponibilité	Must	3	Ghofrane
US#6	Contraintes de transport	Should	5	Ghofrane
US#7	Génération automatique du planning	Must	8	Wissem
US#8	Ajustement dynamique du planning	Could	8	Wissem
US#9	Visualisation du planning	Must	3	Alae
US#10	Statistiques de productivité	Should	5	Alae



3.Sprint Backlogs

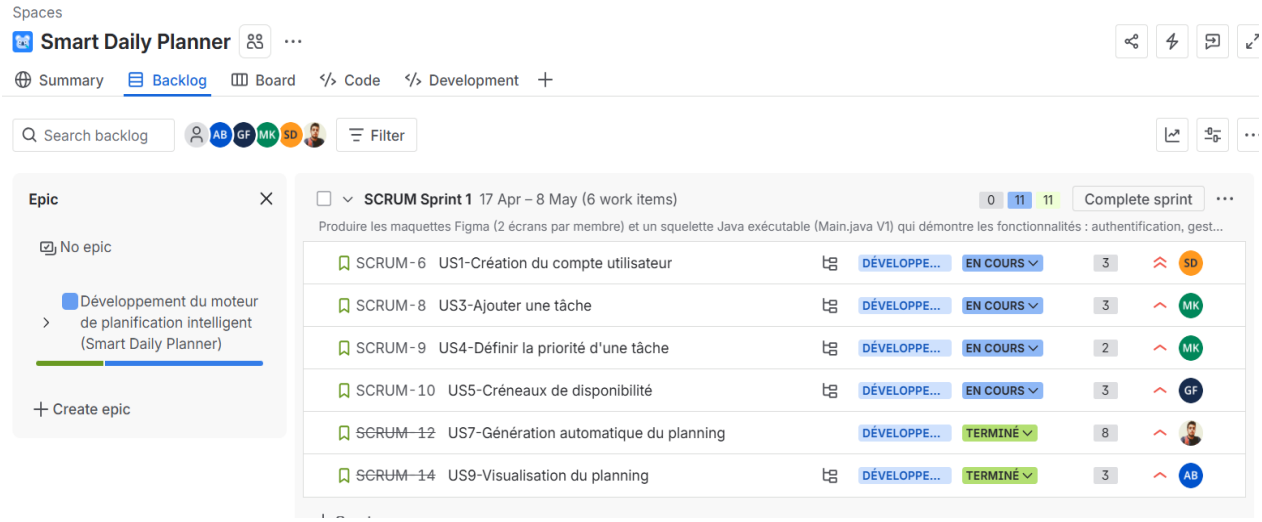
3.1 sprint planning

Sprint 1 Goal: Concevoir l'expérience utilisateur (maquettes Figma) et produire l'architecture Java exécutable

Durée: 1 semaine

US Sélectionnées: US1, US2, US3, US4 (Must) + US9, US10 (Must)

Definition of Done: Code compile | Main.java s'exécute | PR avec review | Maquettes Figma



3.2 Sprint2 planning

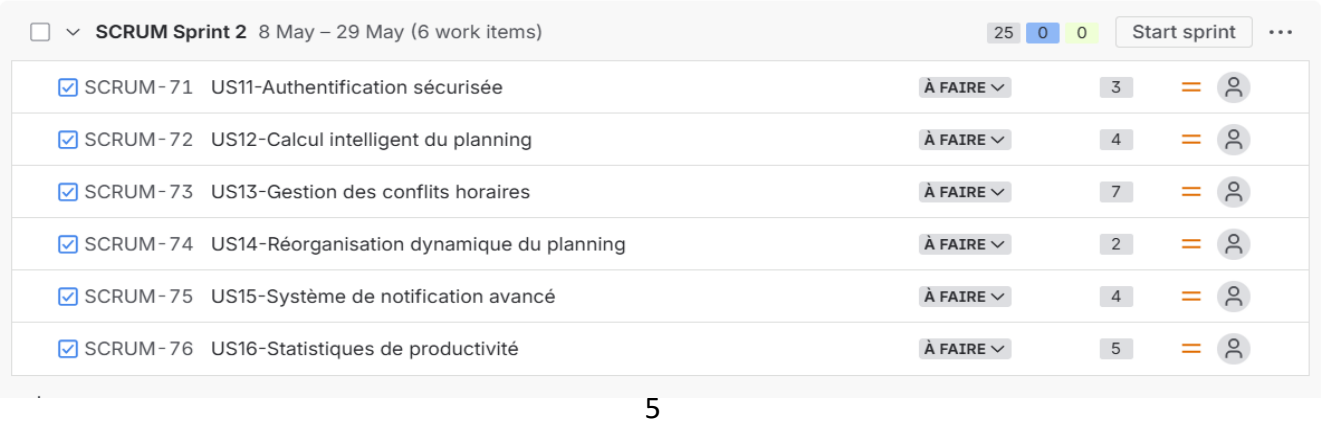
Sprint 2 Goal: Livrer un prototype complet avec logique métier et rapport consolidé

Durée: 1 semaine

US Sélectionnées: US5, US6, US7, US8 + finition US1-4, US9-10

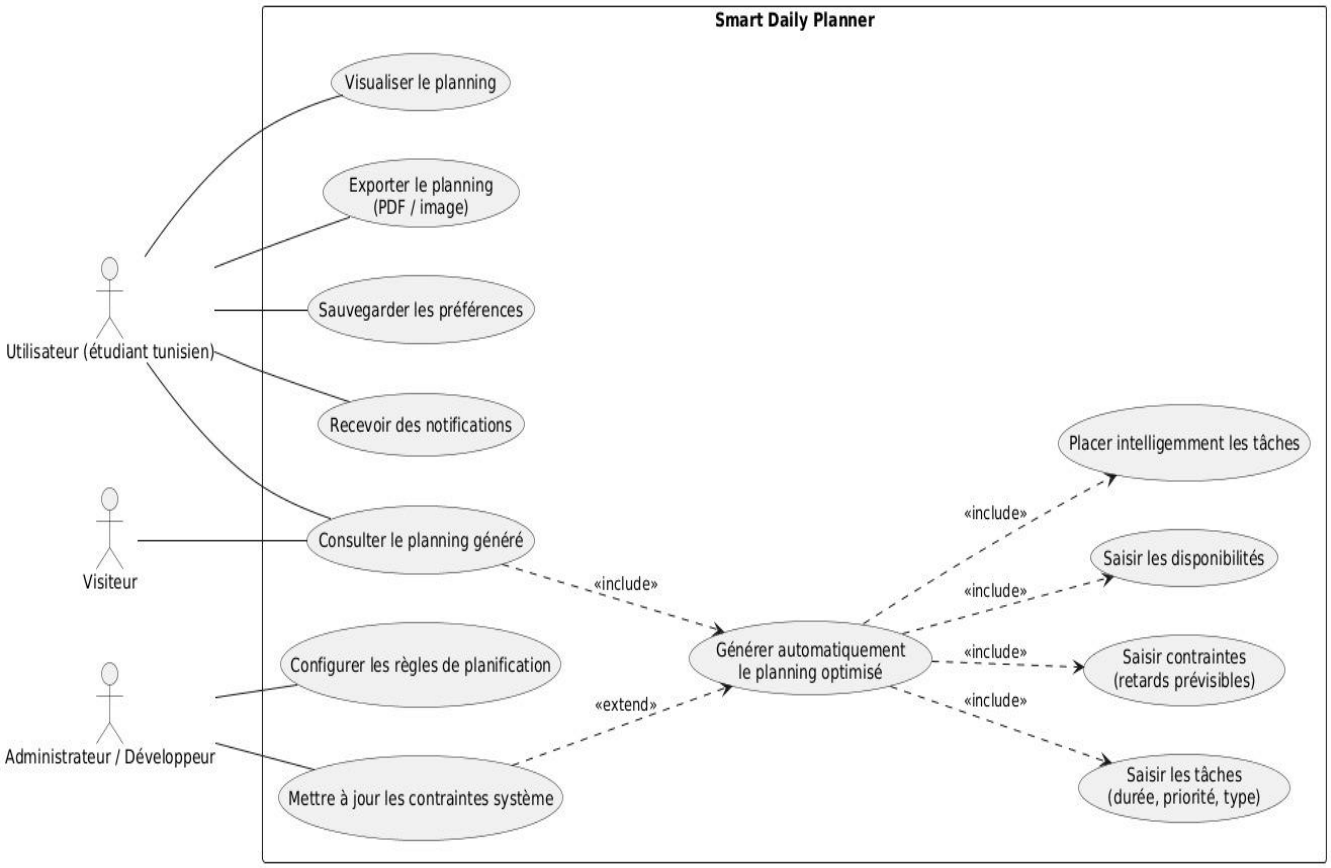
Vélocité Sprint 1: Estimée: 40 points | Réalisée: 35 points

Definition of Done: Logique métier réelle | Classes interagissent | Rapport PDF (15-20 p) | Tests réussis



4.Modelisation UML

4.1 Diagramme de Cas d'Utilisation (DCU)



3 acteurs : Utilisateur (étudiant), Visiteur, Administrateur / Développeur.

12 cas d'utilisation : 1. Visualiser le planning 2. Exporter le planning (PDF / image) 3. Sauvegarder les préférences 4. Recevoir des notifications 5. Consulter le planning généré 6. Générer automatiquement le planning optimisé 7. Placer intelligemment les tâches 8. Saisir les disponibilités 9. Saisir contraintes (retards prévisibles) 10. Saisir les tâches (durée, priorité, type) 11. Configurer les règles de planification 12. Mettre à jour les contraintes système

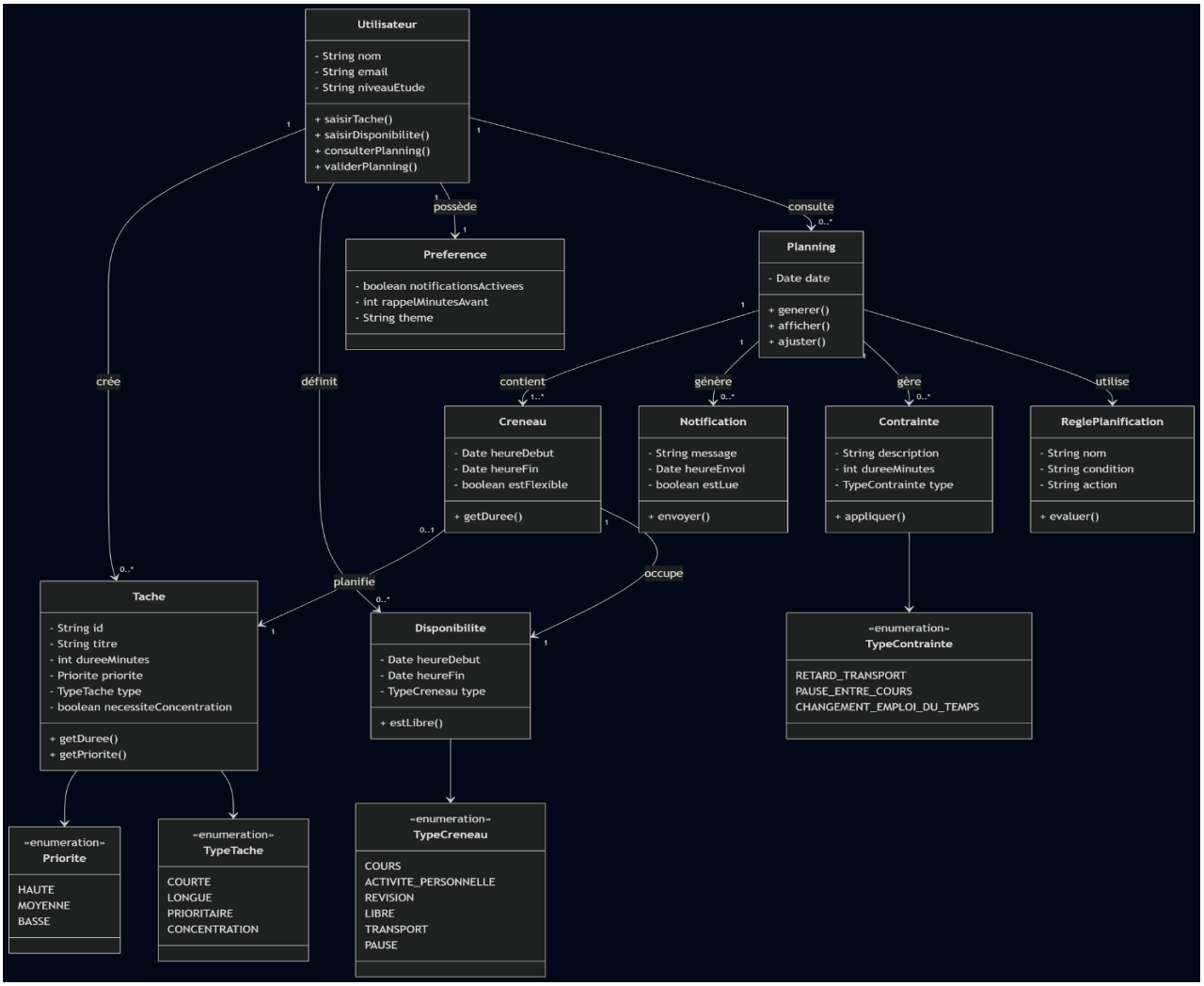
Relations : * **<<include>>** : La génération automatique inclut obligatoirement la saisie des tâches, disponibilités, contraintes et le placement intelligent.

<<extend>> : La mise à jour des contraintes système peut étendre la génération du planning.

Héritage : Partage de l'accès à la consultation entre Visiteur et Utilisateur.

Labels en français Fichier : DCU_SmartDailyPlanner.puml

Diagramme de Classes (DC)



Membres : Samar : Utilisateur, Administrateur (héritent de *Personne*), Preference.

Maamoun : Tache, Disponibilite, Contrainte.

Ghofrane : MoteurAnalyse (ReglePlanification).

Wissem : MoteurPlanning (Creneau).

Alae : Planning, TableauBord, Statistique.

Énumérations : Priorite, TypeTache, TypeCreneau, TypeContrainte.

Relations :

Héritage : Utilisateur et Administrateur vers la classe abstraite Personne.

Composition/Agrégation : Planning contient des Créneaux et gère les Notifications.

Association : Utilisateur possède Preference ; Planning utilise ReglePlanification.

Cardinalités : 1..1, 0.., *1*...

Spécificités techniques :

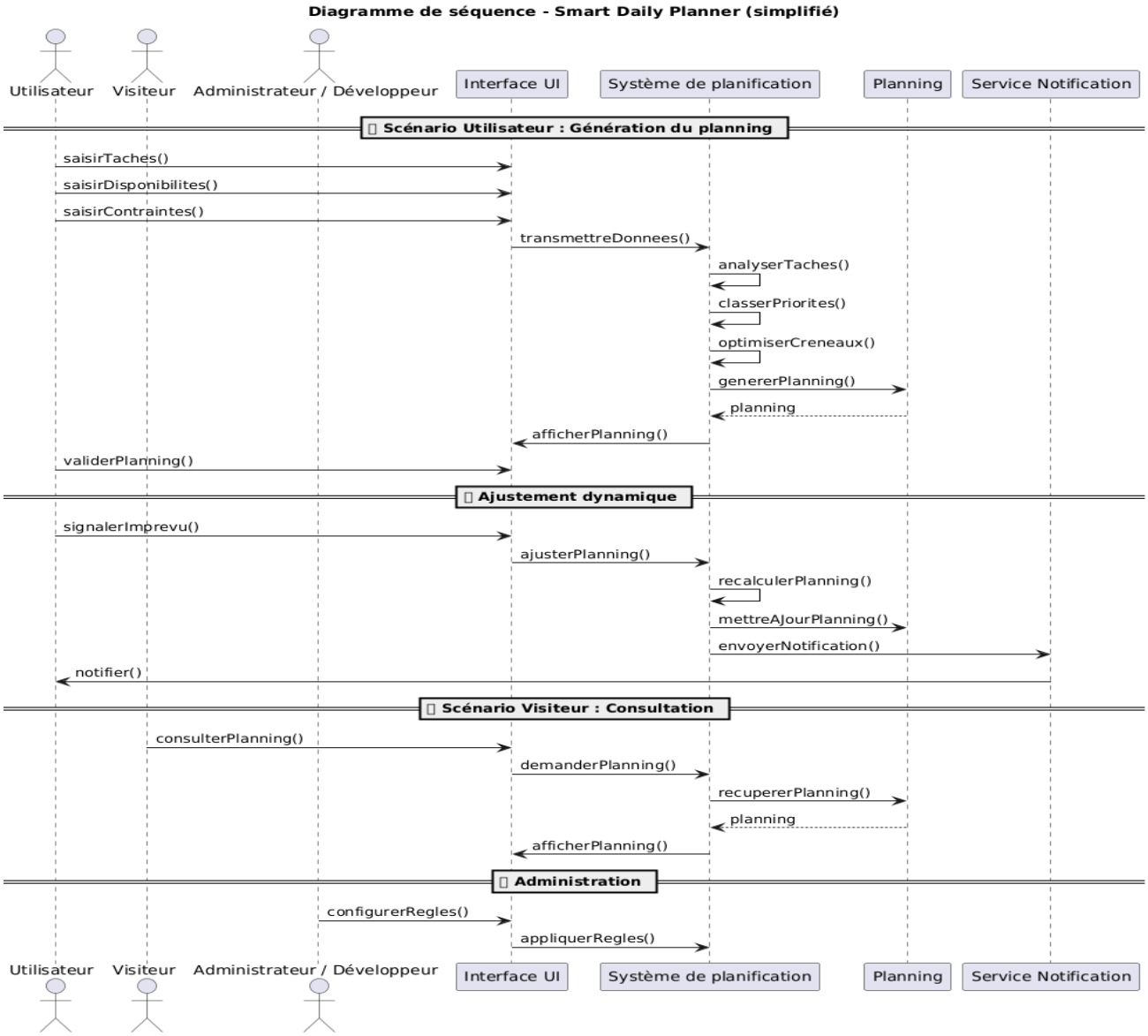
Encapsulation : Attributs privés (–), méthodes publiques (+).

Méthodes métier : `generer()`, `appliquer()`, `evaluer()`, `validerPlanning()`.

Labels : Français intégral.

Fichier : DC_SmartDailyPlanner.puml

Diagrammes de Séquence (DS)



Scénarios représentés :

1.Génération du Planning (Principal) : Saisie des données (tâches, dispos, contraintes), analyse, optimisation des créneaux et affichage final.

2.Ajustement dynamique : Gestion des imprévus, recalcul et envoi de notifications.

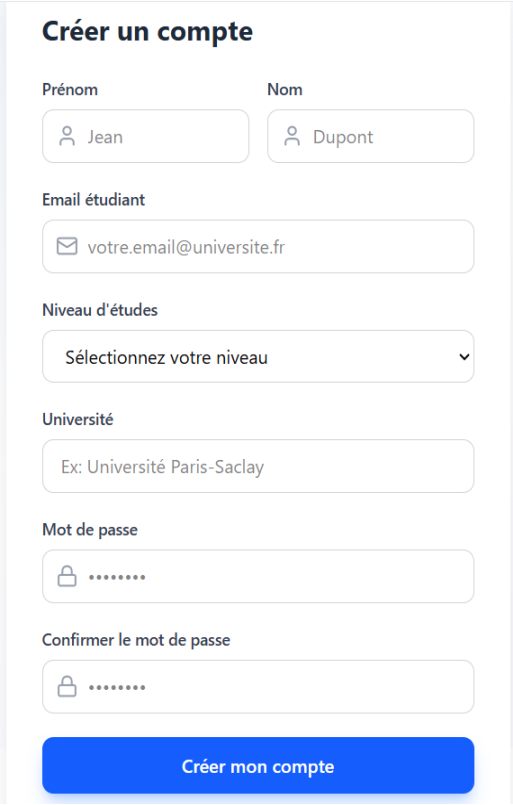
3.Consultation (Visiteur) : Accès en lecture seule au planning.

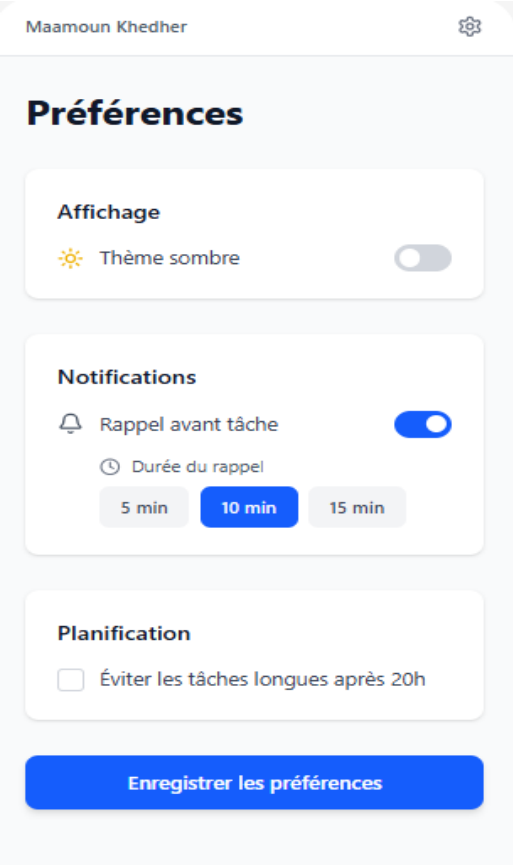
4.Administration : Configuration et application des règles système.

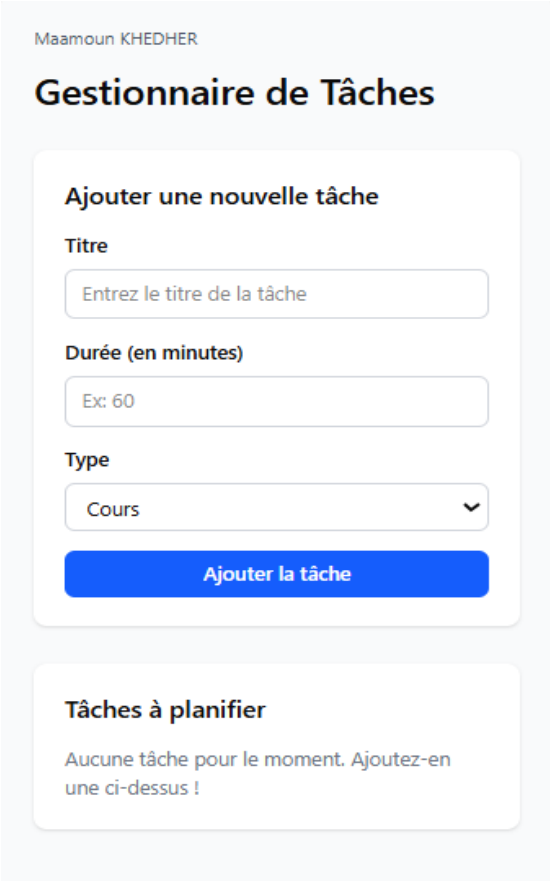
Messages synchrones avec retours

Fichiers: DS_SmartDailyPlanner_Scenario1.puml, DS_SmartDailyPlanner_Scenario2.puml

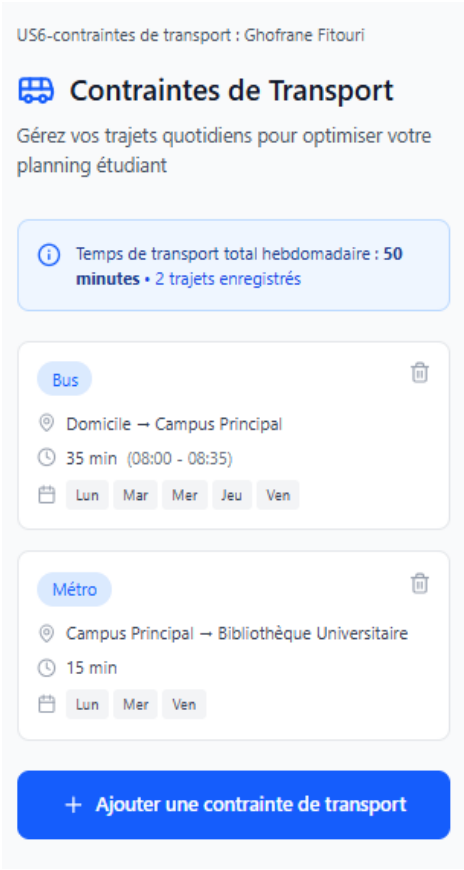
4. MAQUETTES FIGMA (UI/UX)

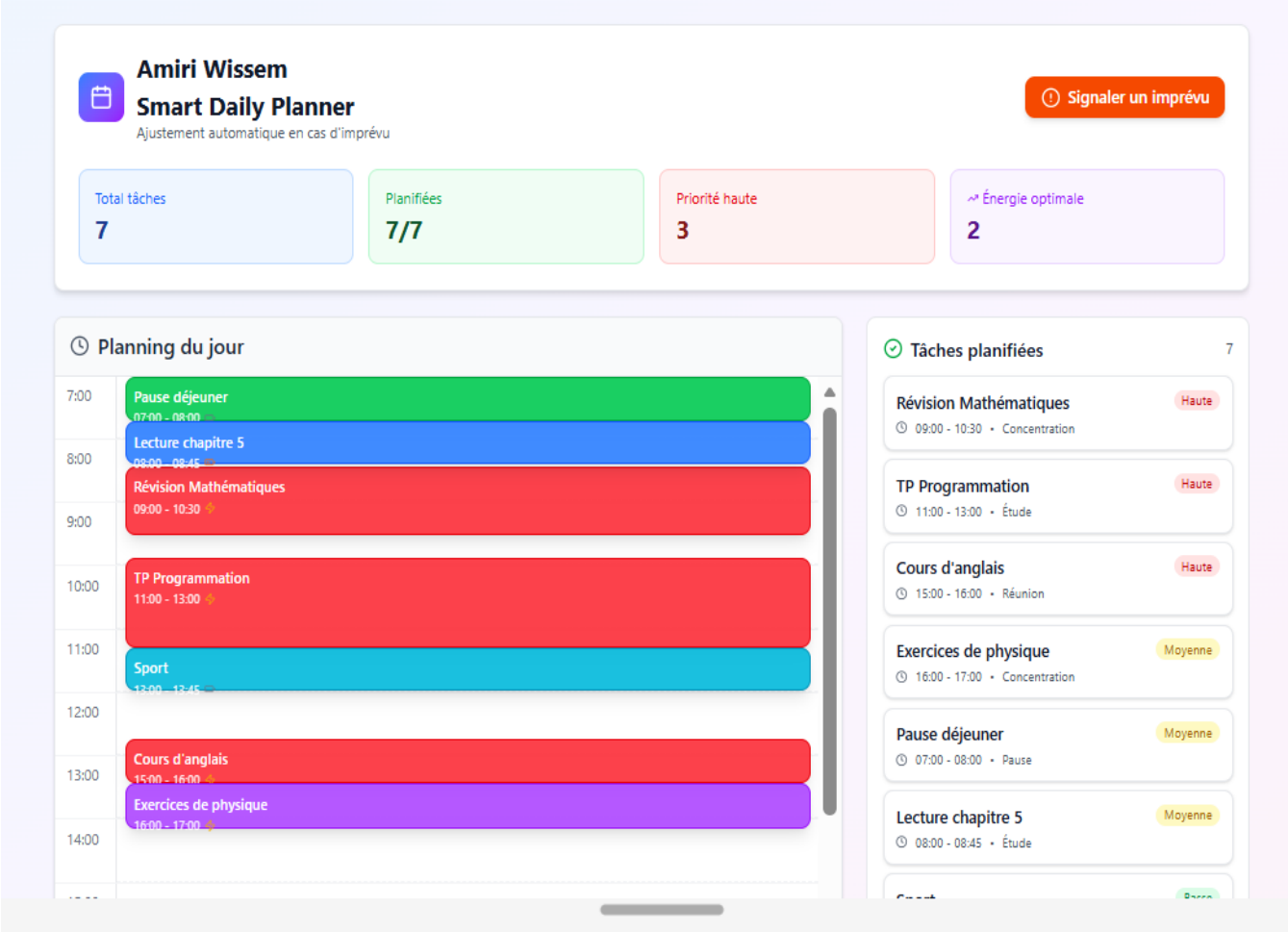






US5-créneaux de disponibilité :
Ghofrane Fitouri
Planification de Temps - Étudiants
Tout effacer
Sauvegarder
Sélectionnez vos créneaux de disponibilité
Cliquez et glissez pour sélectionner plusieurs créneaux
Lundi
Mardi
Mercredi
Jeudi
Vendr
8:00
9:00
10:00
11:00
12:00
13:00
14:00


US6-contraintes de transport : Ghofrane Fitouri
Contraintes de Transport
Gérez vos trajets quotidiens pour optimiser votre planning étudiant
Temps de transport total hebdomadaire : 50 minutes • 2 trajets enregistrés
Bus
Domicile → Campus Principal
35 min (08:00 - 08:35)
Lun Mar Mer Jeu Ven
Métro
Campus Principal → Bibliothèque Universitaire
15 min
Lun Mer Ven
Ajouter une contrainte de transport


Amiri Wissem
Smart Daily Planner
Ajustement automatique en cas d'imprévu
Signaler un imprévu
Total tâches
7
Planifiées
7/7
Priorité haute
3
Énergie optimale
2
Planning du jour
7:00
8:00
9:00
10:00
11:00
12:00
13:00
14:00
Pause déjeuner
07:00 - 08:00
Lecture chapitre 5
08:00 - 08:45
Révision Mathématiques
09:00 - 10:30
TP Programmation
11:00 - 13:00
Sport
13:00 - 13:45
Cours d'anglais
15:00 - 16:00
Exercices de physique
16:00 - 17:00
Tâches planifiées
7
Révision Mathématiques
Haute
09:00 - 10:30 • Concentration
TP Programmation
Haute
11:00 - 13:00 • Étude
Cours d'anglais
Haute
15:00 - 16:00 • Réunion
Exercices de physique
Moyenne
16:00 - 17:00 • Concentration
Pause déjeuner
Moyenne
07:00 - 08:00 • Pause
Lecture chapitre 5
Moyenne
08:00 - 08:45 • Étude

Amiri Wissem

Smart Daily Planning

Générez automatiquement votre planning optimisé en fonction de vos tâches, priorités et niveaux d'énergie

Ajouter une tâche

Nom de la tâche

Ex: Réviser mathématiques

Durée (minutes)

60

Priorité

Moyenne

☐ Nécessite une forte concentration

Ajouter la tâche

Configuration des créneaux horaires

Masquer

15h - 16h	☑ Disponible	Énergie Élevée
16h - 17h	☑ Disponible	Énergie Élevée
17h - 18h	☑ Disponible	Énergie Moyenne
18h - 19h	☑ Disponible	Énergie Moyenne
19h - 20h	☑ Disponible	Énergie Faible
20h - 21h	☐ Disponible	Énergie Faible

Visualisation du planning

US#9 - Membre 5 - Alae Benelmekki

Voir les statistiques

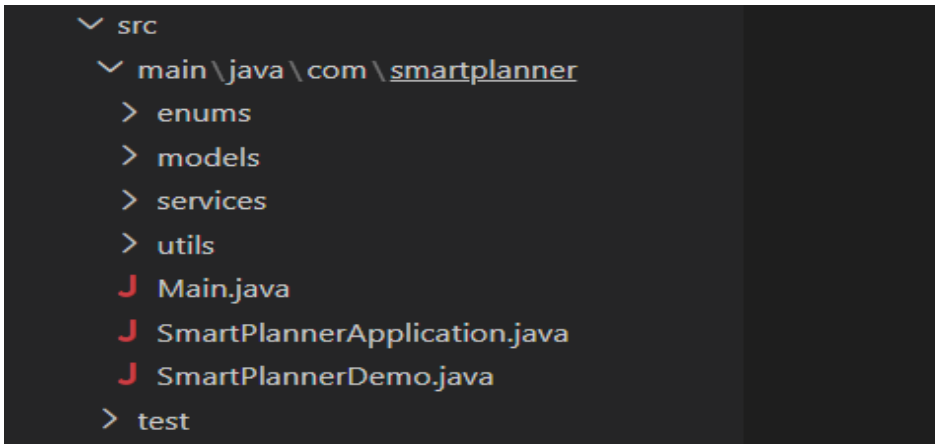
Vue Jour | Vue Semaine

Rechercher une tâche... | Actualiser | Exporter PDF | Filtrer par date

Horaire	Lundi	Mardi	Mercredi	Jeudi	Vendredi
8h-9h	Trajet Tunis-Campus	Trajet Sousse-Campus	Trajet vers campus		Trajet vers campus
9h-10h	Mathématiques - Prof. Ahmed Ben Ali	Préparation examen	Électronique - Prof. Karim Bouaziz	Révision générale	Chimie - Prof. Imen Gharbi
10h-11h	Mathématiques - Prof. Ahmed Ben Ali	Informatique - Prof. Mohamed Trabelsi	Électronique - Prof. Karim Bouaziz	Révision générale	Chimie - Prof. Imen Gharbi
11h-12h	Pause café	Informatique - Prof. Mohamed Trabelsi	TP Électronique	Pause	TP Chimie
12h-13h	Révision algèbre	Déjeuner	Déjeuner	Gestion de projet - Prof. Nadia Karray	Déjeuner
13h-14h	Déjeuner	Projet Python	Mathématiques avancées	Déjeuner	Préparation semaine prochaine
14h-15h	Physique - Prof. Sarra Mansouri	Projet Python	Mathématiques avancées		Préparation semaine prochaine
	Physique - Prof. Sarra				

6. CODE - APPROCHE HYBRIDE AGL + IA

6.1 Architecture du Code



6.2 Approche Hybride pour Chaque Membre

Membre	Rôle	Fonctionnalité	Classes	Méthodes clés	Statut
Samar Dhouib	Product Owner + Dev	Authentification & Profil	`Utilisateur`, `Preference`	`creerUtilisateur()`, `authentifierUtilisateur()`, `configuerProfil()`	**Terminé** (logique complète)
Maamoun Khedher	Scrum Master + Dev	Gestion des Tâches	`Tache`, `Disponibilite`, `Contrainte`	`ajouterTache()`, `modifierTache()`, `marquerTacheFaite()`	**Terminé** (logique complète)

Ghofrane Fitouri	Développeur	Moteur d'Analyse	`MoteurAnalyse` (dans services)	`analyserComportement()`, `detecterNiveauEnergie()`, `classifierTache()`	**Terminé** (logique complète)
Wissem Amiri	Développeur	Moteur de Planning	`MoteurPlanning` (dans services)	`genererPlanning()`, `optimiserCreneau()`, `gererImprevus()`	**Terminé** (logique complète)
Alae Benel mekki	Développeur Dashboard & Statistiques	Dashboard & Statistiques	`TableauBord`, `Statistique`, `ElementPlanning`	`afficherPlanning()`, `afficherStatistiques()`, `calculerTauxCompletion()`	**Terminé** (logique complète)

6.3 Processus IA Assistée (Exemple)

Alae (developpeur :Tableau de Bord & Statistiques :US9+10) J'ai utilisé l'IA pour générer certaines méthodes complexes. Voici les prompts que j'ai formulés (le code obtenu a été intégré et adapté dans les classes *TableauBord*, *Statistique* et *Main*).

calculerTauxCompletion()	*"Calcule le taux de complétion (terminées/total × 100) en évitant la division par zéro."*
afficherStatistiques()	"Affiche les statistiques avec une barre de progression (ex: *[[redacted] 40%*)."
marquerTacheFaite()	"Écris une méthode *marquerTacheFaite(String idTache)* qui change le statut d'une tâche à 'TERMINÉE' et met à jour le taux de complétion."
afficherTachesParPriorite()	"Ajoute une méthode *afficherTachesParPriorite()* qui liste les tâches par ordre décroissant de priorité."
Mise a jour automatique du planning apres ajout d'une tache	"Lorsqu'on ajoute une tâche, si un planning existe déjà, régénère automatiquement le planning avec les nouvelles données et affiche-le. Gère les cas où la régénération échoue."

Wissem (developpeur : Planificateur Intelligent:US7+8)

US7-Génération automatique du planning	En tant qu'utilisateur, je veux cliquer sur "Générer mon planning", afin de voir un emploi du temps optimisé qui place toutes mes tâches. Critères d'acceptation :

	1. Le planning généré respecte : créneaux disponibles, durées des tâches, priorités. 2. Les tâches "concentration" sont placées dans les créneaux où l'énergie est "Élevée". Le planning s'affiche sous forme de tableau horaire (8h → 20h)
US8-Ajustement dynamique du planning :	je veux que mon planning s'ajuste automatiquement en cas d'imprévu, afin de garder un emploi du temps cohérent et optimisé. ➢ Critères d'acceptation : 1. Le système détecte ou reçoit un imprévu (retard, changement d'horaire, nouvelle tâche). 2. Le planning est recalculé automatiquement sans perdre les tâches importantes. 3. Les contraintes sont respectées (disponibilités, durées, priorités). 4. Les tâches prioritaires restent planifiées en premier. 5. Les tâches de type "concentration" restent dans les créneaux d'énergie élevée si possible. 6. Le planning mis à jour est affiché immédiatement. Une notification informe l'utilisateur du changement

7. TABLEAU COMPARATIF AGL vs IA

Critère	AGL Pur (Sprint 1)	Hybride AGL + IA (Sprint 2)
Cohérence Structurelle	Garantie par DC	Vérifiée par review
Rapidité d'Implémentation	Méthodes vides	Rapide avec corrections
Reproductibilité	Déterministe	Dépend du prompt
Qualité du Code Métier	Pas de logique réelle	Logique vérifiée
Maintenabilité	Structure claire	Code commenté
Temps d'Apprentissage	Lent (AGL manuel)	Rapide (IA guide)
Satisfaction Utilisateur	Prototype limité	Prototype fonctionnel

→ L'approche hybride AGL + IA offre le meilleur compromis entre qualité, rapidité et apprentissage.

8. SPRINT REVIEWS & RÉTROSPECTIVES

8.1 Sprint 1 Review

Quelles US sont Done? US1, US2, US3, US4,US5,US7,US8,US9,US10
Quelles US sont reportées? US6 (Moteurs IA - nécessitent plus de logique)
Le PO est-il satisfait? Oui - Maquettes cliquables + squelette Java fonctionnant
Maquettes correspondent-elles au besoin? À vérifier cette semaine - écrans Figma à finir

8.2 Sprint 1 Rétrospective

Ce qui a bien fonctionné:

Coordination agile - communication quotidienne fluide

AGL + IA - génération rapide du squelette depuis DC

Jira bien configuré - suivi transparent du projet

Ce qui doit être amélioré:

Maquettes Figma - à finaliser rapidement cette semaine

Documentation des prompts IA - à structurer mieux

8.3 Sprint 2 Planning (À faire)

Sprint 2 Goal: Livrer logique métier complète + rapport consolidé + préparer soutenance

Priorités:

1. Compléter maquettes Figma (10 écrans, navigation cliquable)
2. Implémenter logique métier complète (US5-8)
3. Finir rapport PDF (15-20 pages)
4. Préparer démo soutenance (10 min pitch)

9. BILAN INDIVIDUEL

Alae Benelmekki (developpeur)

J'ai co-construit le Product Backlog, rédigé mes deux user stories (US#9, US#10) et découpé mes sous-tâches. J'ai développé le module complet (classes `TableauBord`, `Statistique`, `ElementPlanning`, `Main.java` coloré avec barre de progression), testé le code et corrigé les erreurs. J'ai poussé l'ensemble sur GitHub via une branche dédiée (commits, Pull Request). J'ai réalisé deux maquettes Figma navigables (la première affiche la vue semaine). J'ai rédigé l'intégralité du rapport final et participé au sprint planning.

J'ai également exploré une extension mobile du projet : génération automatique d'une page HTML responsive du planning avec QR code. Cette fonctionnalité, bien que non intégrée au livrable final, démontre ma capacité à anticiper les besoins utilisateurs et à proposer des solutions innovantes *Ce projet m'a appris à livrer une fonctionnalité complète de A à Z.*

Wissem Amiri (developpeur)

Je code les classes Planning, ElementPlanning, PlanningService (avec genererPlanning, assignerTachesAuxCreneaux, et NotificationService)Je modélise dans le diagramme de classes mes classes Planning, Creneau et Notification.Je définis dans le diagramme de cas d'utilisation les cas « Générer automatiquement le planning optimisé » et « Placer intelligemment les tâches ».Je modélise dans les diagrammes de séquence les scénarios de génération du planning et d'ajustement dynamique.Je contribue sur GitHub : création du dépôt avec .gitignore,.Je travaille sur la branche feature/smart-scheduler.

Samar Dhouib (PO)

En tant que Product Owner du projet Smart Daily Planner, j'ai participé à la définition des besoins fonctionnels et à l'organisation du travail de l'équipe. J'ai rédigé deux User Stories avec leurs sous-tâches et j'ai également préparé le Sprint 2 en planifiant les tâches à réaliser . J'ai conçu deux maquettes sur Figma afin de représenter l'interface de l'application et faciliter la compréhension du projet. Ce travail m'a permis d'utiliser Jira pour la première fois, notamment pour gérer le backlog, suivre l'avancement des tâches et organiser les sprints. J'ai aussi appris l'importance du travail en groupe, de la communication et de la coordination entre les membres. De plus, cette expérience m'a aidé à mieux gérer mon temps et à respecter les délais fixés. Malgré quelques difficultés d'adaptation aux nouveaux outils, cette expérience a été très enrichissante sur les plans technique et organisationnel

Maamoun Khedher (SM)

En tant que Scrum Master, j'ai mis en place le dépôt GitHub et le tableau Kanban, animé les cérémonies Scrum et veillé aux revues de code. La coordination des deux sprints a été exigeante, notamment pour relancer les maquettes Figma et ajuster la planification. Côté développement, j'ai livré le module de gestion des tâches (CRUD, priorités) en assurant son intégration avec le

planificateur. J'ai appris l'impact décisif d'un Daily Scrum bien mené pour débloquer l'équipe rapidement. La principale difficulté a été de bien assumer mon rôle de facilitateur,

Ghofrane Fitouri (dev)

En tant que développeuse J'ai contribué au développement du logiciel de en utilisant Java pour la programmation, GitHub pour la gestion du code source et Jira pour le suivi des tâches. Mon rôle consistait à identifier les périodes de productivité optimale afin d'adapter les créneaux de travail proposés par l'application J'ai participé à la conception des fonctionnalités principales comme la gestion des emplois du temps et des priorités. Grâce à ce projet, j'ai appris à travailler en équipe, à gérer les versions du code et à organiser un projet de manière structurée. J'ai également amélioré mes compétences en programmation orientée objet et en résolution de problèmes. L'utilisation de Jira m'a aidé à mieux comprendre la gestion de projet agile. Parmi les difficultés rencontrées, il y avait la gestion des conflits de version sur GitHub et la compréhension de certaines fonctionnalités avancées en Java. J'ai aussi eu des défis pour bien organiser le temps et répartir les tâches efficacement.